# Capítulo 2: Fundamentos del Lenguaje WGSL

## Sintaxis moderna estilo Rust, tipado estricto, estructuras y la física del alineamiento

**Serie:** *BrowserOS: Computación Nativa en la Web con Wasm, WebGPU y WGSL* | **Autoría:** Antigravity Engineering Lab
**Referencias en O'Reilly:** Programming Rust, Programming with Types y GPU-Accelerated Computing with Python 3 and CUDA

### 1. Filosofía de Diseño: Por qué la Web Exige WGSL

Para llevar computación de alto rendimiento al navegador sin comprometer la seguridad del sandbox, el consorcio W3C descartó adaptar lenguajes fragmentados del pasado (GLSL, HLSL o binarios SPIR-V). La web requería un lenguaje estándar en texto plano que satisficiera tres condiciones no negociables:

1. **Verificación Formal en Compilación:** Validado estrictamente por el navegador (mediante los compiladores de código abierto *Tint* en Chromium y *Naga* en Firefox) antes de generar código para Vulkan, Metal o DirectX 12.
2. **Seguridad de Memoria por Construcción:** Imposibilidad de desbordar búferes (**out-of-bounds memory safety**) mediante sujeción automática o límites tipados.
3. **Determinismo Absoluto:** Eliminación de comportamientos indefinidos dependientes del fabricante de la GPU.

Como argumentan Jim Blandy y Jason Orendorff en **Programming Rust**, los lenguajes de sistemas modernos aplican el axioma de **«corrección por construcción»**: los errores de memoria deben ser imposibles de compilar en lugar de degenerar en fallos silenciosos en producción. WGSL traslada este principio al silicio gráfico.

### 2. Sintaxis y Mutabilidad: `let`, `var`, `const` y `override`

WGSL adopta la semántica de inmutabilidad por defecto inspirada en Rust. Confundir los cuatro modificadores es el error más recurrente en desarrolladores que provienen de C o JavaScript:

| Declaración | Mutabilidad | Momento de Evaluación | Uso Arquitectónico en BrowserOS |
|---|---|---|---|
| `const` | Inmutable | Compilación del Shader | Constantes fijas; no consumen registros ni memoria. |
| `override` | Inmutable | Creación del Pipeline | Parámetros inyectados desde JS al compilar el pipeline. |
| `let` | **Inmutable** | Tiempo de Ejecución | Valores locales intermedios; optimiza registros de GPU. |
| `var` | **Mutable** | Tiempo de Ejecución | Variables reasignables con espacio de memoria explícito. |

**Prohibición Estricta de Recursividad:** En WGSL las funciones (`fn`) no pueden ser recursivas. A diferencia de una CPU con memoria dinámica, cada hilo de GPU cuenta con un banco fijo de registros físicos; una llamada recursiva provocaría un desbordamiento de pila fatal en el chip.

### 3. Tipado Fuerte y Prohibición de Casts Implícitos

En **Programming with Types**, Vlad Riscutia demuestra que un sistema de tipos riguroso actúa como la primera línea de defensa de la integridad de memoria. En WGSL:

- **Cero conversiones automáticas:** Expresiones como `1.0 + 2` o `f32_val + u32_val` arrojan un error de compilación inmediato. El programador está obligado a escribir conversiones explícitas (`1.0 + f32(2)`). Esto previene que instrucciones ocultas de transcodificación ralenticen bucles que ejecutan millones de hilos.
- **Vectores y Swizzling:** Vectores de 2, 3 y 4 dimensiones (`vec2`, `vec3`, `vec4`) con soporte para acceso espacial (`.xyzw`), cromático (`.rgba`) y reordenación arbitraria instantánea (`posicion.zyx`, `vector.xxxx`).

### 4. Estructuras (`struct`) y la Física del Alineamiento a 16 Bytes

Como explican Patterson y Hennessy (**Computer Organization and Design**), los buses de memoria de las GPUs leen bloques continuos de **16 bytes (128 bits)**. Si una estructura de datos no se empaqueta conforme a esta física, se producen desalineamientos críticos:

| Tipo WGSL | Tamaño (Size) | Alineamiento | Comportamiento Físico en Memoria |
|---|---|---|---|
| `f32`, `i32`, `u32` | 4 bytes | 4 bytes | Alineado a múltiplos de 4 (0, 4, 8, 12, 16…). |

| Tipo WGSL | Tamaño (Size) | Alineamiento | Comportamiento Físico en Memoria |
|---|---|---|---|
| `vec2<f32>` | 8 bytes | 8 bytes | Alineado a múltiplos de 8 (0, 8, 16, 24…). |
| **`vec3<f32>`** | **12 bytes** | **16 bytes** | ⚠️ **Exige frontera de 16 bytes. Deja 4B de padding a la derecha.** |
| `vec4<f32>` | 16 bytes | 16 bytes | Alineamiento óptimo nativo de palabra GPU. |
| `mat4x4<f32>` | 64 bytes | 16 bytes | 4 columnas consecutivas de `vec4<f32>`. |

**La Regla de Oro del Empaquetamiento:** Al compartir estructuras complejas entre JavaScript/Wasm y WGSL, evita `vec3<f32>` o rellena manualmente los 4 bytes de padding en la CPU (`Float32Array`). Si omites el padding, la GPU leerá los campos de las siguientes entidades totalmente desfasados, provocando corrupción silenciosa de datos.

## 5. Decoradores: El Contrato del Shader (`@group`, `@binding`, `@compute`)

Los decoradores `@` vinculan el shader con los recursos del sistema host:

- `@compute` y `@workgroup_size(X, Y, Z)`: Definen el punto de entrada y el tamaño del bloque concurrente.
- `@group(G) @binding(B)`: Mapean búferes y texturas desde el `GPUBindGroup` de la CPU al código WGSL.
- `@builtin(...)`: Inyectan variables físicas de la GPU: `global_invocation_id` (coordenada absoluta 3D) y `local_invocation_id` (índice dentro del bloque).

## 6. Taller Práctico: Laboratorio de Structs y Detección de Desfase

Se ha creado el laboratorio interactivo `workshop_02_memory_alignment_structs.html` que ilustra la física del empaquetamiento:

- Modela un struct `ParticulaFisica` de 48 bytes (posicion, velocidad, masa, vida, id_tipo y 4 bytes de padding).
- Permite alternar en tiempo real entre el empaquetamiento correcto (48B) y el empaquetamiento ingenuo sin padding (44B), visualizando la corrupción de memoria en la GPU cuando la CPU desalinea los datos.

## 7. Bibliografía y Lecturas Recomendadas para Profundizar

| Obra y Autoría (O'Reilly) | Aportación Clave al Dominio de WGSL y Tipos de Sistemas |
|---|---|
| **Programming Rust (3rd Ed.)**<br>J. Blandy, J. Orendorff, L. Tindall<br>(O'Reilly Media) | La referencia por excelencia para dominar la semántica de inmutabilidad (`let`), expresiones libres de efectos secundarios y el diseño de software seguro que inspiró la especificación WGSL. |
| **Programming with Types**<br>Vlad Riscutia<br>(Manning Publications) | Tratado teórico-práctico indispensable sobre cómo los tipos de datos compuestos determinan el layout físico en memoria y previenen desbordamientos en bajo nivel. |
| **GPU-Accelerated Computing**<br>N. Cautaerts & H. Ghorbanfekr<br>(Packt Publishing) | Explica la física del acceso coalescente a VRAM y el coste masivo en ciclos de reloj que sufre una GPU cuando los datos de entrada no respetan fronteras de 16 bytes. |
| **The Evolune Metaverse**<br>F. Lisitano & J. Hickie<br>(Apress) | Pionero en la aplicación de WGSL para pipelines de computación espacial en el navegador, detallando el uso de decoradores `@group` y `@binding` en producción. |
| **Computer Organization & Design**<br>D. Patterson & J. Hennessy<br>(Morgan Kaufmann) | Texto canónico sobre arquitectura de computadores. Imprescindible para comprender el diseño de memorias alineadas y las jerarquías de almacenamiento en procesadores SIMD/SIMT. |

## 8. Próximo Paso: Hacia el Capítulo 3

En el **Capítulo 3: Compute Shaders y Jerarquía de Memoria**, profundizaremos en la memoria compartida ultrarrápida en chip (`var<workgroup>`), barreras de sincronización (`workgroupBarrier()`) y la implementación de algoritmos paralelos avanzados como reducciones y multiplicación matricial por bloques (*tiled GEMM*).